

Data Handoff — Francesco → Arthur

Project 11_02 — Near Real-Time Financial Forecasting using Amazon SageMaker

Overview

Francesco's job (steps 7–8) is to manage the DynamoDB table where forecast results and anomaly flags land, and to build the QuickSight dashboard.

Arthur (steps 9–10) reads anomaly flags from DynamoDB and sends alerts via SNS, plus orchestrates the whole pipeline with Step Functions and CloudWatch.

The handoff between Francesco and Arthur is essentially: **Francesco exposes the DynamoDB table and its anomaly flag, Arthur listens for changes and fires alerts.**

Context: Who writes to the DynamoDB table?

To avoid confusion: the table is **owned by Francesco** (he creates it, defines the schema, manages it), but **Ralph's Lambda is the one writing to it** after each forecast cycle. Arthur's Lambda only reads from it.

```
Ralph (writes)  ->  DynamoDB (Francesco owns)  ->  Arthur (reads + alerts)
                |
                +-->  Francesco (reads for dashboard widgets)
```

What Francesco gives Arthur

1. DynamoDB table info

Property	Value
Table name	<code>proj1102-forecasts</code>
Region	(agreed AWS region — e.g. <code>us-east-1</code>)
Partition key	<code>ticker</code> (string)
Sort key	<code>timestamp</code> (string ISO-8601)
Stream	Enabled with <code>NEW_IMAGE</code> view type
Table ARN	(Francesco shares once created)
Stream ARN	(Francesco shares once created)

2. Schema (what each row looks like)

Attribute	Type	Example	Notes
<code>ticker</code>	string	<code>AAPL</code>	Partition key
<code>timestamp</code>	string	<code>2026-04-18T10:00:00Z</code>	Sort key, ISO-8601 UTC
<code>p10</code>	number	<code>183.10</code>	Lower bound
<code>p50</code>	number	<code>185.42</code>	Predicted price
<code>p90</code>	number	<code>187.80</code>	Upper bound

Attribute	Type	Example	Notes
actual	number	192.10	Real price at that time
is_anomaly	boolean	true	This is what triggers Arthur's alert
delta	number	+6.68	actual - p50
severity	string	high	low / medium / high based on how far outside the band

How Arthur subscribes — DynamoDB Streams

Recommended approach: **DynamoDB Streams + Lambda trigger**. Whenever Ralph writes a new row (or updates one), DynamoDB Streams fires Arthur's Lambda automatically — no polling needed.

Setup

1. Francesco enables Streams on `proj1102-forecasts` with view type `NEW_IMAGE`.
2. Arthur creates a Lambda `proj1102-anomaly-alert`.
3. Arthur attaches the DynamoDB Stream as the Lambda trigger (batch size 10, starting position LATEST).

What Arthur's Lambda receives (sample event)

```
{
  "Records": [
    {
      "eventID": "abc123",
      "eventName": "INSERT",
      "dynamodb": {
        "Keys": {
          "ticker": { "S": "AAPL" },
          "timestamp": { "S": "2026-04-18T14:00:00Z" }
        },
        "NewImage": {
          "ticker": { "S": "AAPL" },
          "timestamp": { "S": "2026-04-18T14:00:00Z" },
          "p10": { "N": "183.10" },
          "p50": { "N": "185.42" },
          "p90": { "N": "187.80" },
          "actual": { "N": "192.10" },
          "is_anomaly": { "BOOL": true },
          "delta": { "N": "6.68" },
          "severity": { "S": "high" }
        }
      }
    }
  ]
}
```

What Arthur's Lambda does

```
for record in event["Records"]:  
    new_image = record["dynamodb"]["NewImage"]  
    if new_image["is_anomaly"]["BOOL"] == True:  
        publish_to_sns(new_image)
```

That's it. Filter on `is_anomaly == true`, ignore the rest.

SNS message format Arthur publishes

When an anomaly is detected, Arthur publishes a message to the SNS topic `proj1102-alerts`. Subscribers (team emails / SMS) receive a notification.

Suggested message

Subject:

[ALERT] Anomaly detected for AAPL

Body:

Ticker: AAPL
Timestamp: 2026-04-18T14:00:00Z
Predicted: 185.42 (range 183.10 - 187.80)
Actual: 192.10
Delta: +6.68
Severity: high

Full forecast row in DynamoDB: `proj1102-forecasts` (`ticker=AAPL`, `timestamp=2026-04-18T14:00:00Z`)
Dashboard: <QuickSight URL Francesco provides>

QuickSight dashboard URL

Francesco shares the dashboard URL once it's published. Arthur can include it at the bottom of every alert message so recipients have a one-click link to investigate.

<https://eu-west-1.quicksight.aws.amazon.com/sn/dashboards/<dashboard-id>>

Full data flow (summary)

Ralph (SageMaker Lambda)

```
--> writes row to DynamoDB:proj1102-forecasts  
    |  
    v  
DynamoDB Streams (Francesco enables this)  
    |  
    v  
Arthur's Lambda (proj1102-anomaly-alert)  
    |
```

```

|--> if is_anomaly == true
|
+--> SNS topic: proj1102-alerts
|
+--> Email subscribers
+--> SMS subscribers

```

What Arthur can start building now (with dummy data)

Arthur does not need to wait for the full pipeline. He can manually insert a fake row into `proj1102-forecasts` (once Francesco creates the table) with `is_anomaly: true` to fire his Lambda and test the SNS alert end to end.

Dummy row to insert (via AWS CLI or console):

```

{
  "ticker":      {"S": "AAPL"},
  "timestamp":   {"S": "2026-04-18T14:00:00Z"},
  "p10":         {"N": "183.10"},
  "p50":         {"N": "185.42"},
  "p90":         {"N": "187.80"},
  "actual":      {"N": "192.10"},
  "is_anomaly":  {"BOOL": true},
  "delta":       {"N": "6.68"},
  "severity":    {"S": "high"}
}

```

Insert this row → Stream fires → Arthur's Lambda runs → SNS sends test email. If that works, the alerting path is done.

Things to freeze between Francesco and Arthur (agree before coding)

Item	Agreed value
DynamoDB table name	<code>proj1102-forecasts</code>
Partition key	<code>ticker</code>
Sort key	<code>timestamp</code>
Stream view type	<code>NEW_IMAGE</code>
SNS topic name	<code>proj1102-alerts</code>
Anomaly flag attribute	<code>is_anomaly</code> (boolean)
Severity attribute	<code>severity</code> (low / medium / high)
Region	(agreed)
Email/SMS subscribers	The 5 team members + professor (optional)

Any change to the table name, key names, or attribute names must be announced to the group immediately — Ralph (writer), Francesco (dashboard), and Arthur (alerts) all depend on this contract.

Quick checklist

Francesco's side

- ☐ DynamoDB table `proj1102-forecasts` created in agreed region
- ☐ Partition key `ticker`, sort key `timestamp`
- ☐ DynamoDB Streams enabled (`NEW_IMAGE`)
- ☐ Table ARN + Stream ARN shared with Arthur and Ralph
- ☐ QuickSight dashboard URL shared with Arthur

Arthur's side

- ☐ SNS topic `proj1102-alerts` created
- ☐ Team emails subscribed and confirmed
- ☐ Lambda `proj1102-anomaly-alert` created with DynamoDB Stream trigger
- ☐ Filter: only act on `is_anomaly == true`
- ☐ SNS publish formatted with `ticker`, `timestamp`, `predicted range`, `actual`, `severity`
- ☐ Tested end to end with dummy DynamoDB row
- ☐ Lambda + SNS ARNs shared with Ralph (Step Functions wiring later)